A Complete Single-Bit Flip Algorithm for Microscaling (MX) Tensors

Peeyush Agarwal

CS496 Undergraduate Project, Indian Institute of Technology Kanpur

26 October 2026

Abstract

This note gives the full algorithm for deciding what a single stored bit flip does to an MX-quantised network, and explains every step. A flip turns into a wrong prediction only if five things happen in a row: the bit really changes, the stored number changes, no check at read time notices, the datapath carries the error forward, and the top-1 answer moves. The algorithm follows that chain stage by stage. At the encoding stage it sorts every flip into exactly one named case: eight for the shared scale (S0–S7) and fourteen for an element (E0–E7 with sub-cases). The cases are checked in a fixed order, and an exhaustive test over every code, every bit and every scale byte of all five element formats confirms that each flip lands in exactly one case. Each stage is backed by the exhaustive ResNet8 weight campaigns (483,904 faults in `e3m2`, 638,624 in `e4m3`), by re-injection under other datapaths, and by exact counting on real activations.

Contents

1 Why an algorithm, and what "every case" means **1**

2 Background: what is stored **1**

3 The five-stage chain **2**

4 The algorithm **2**

5 Stage 0: the fault model **2**

6 Stage 1: the scale case tree **4**

7 Stage 1: the element case tree **5**

8 Stage 2: free checks and guards **6**

9 Stage 3: the datapath **7**

10 Stage 4: the decision **8**

11 Stage 5: aggregate and protect **8**

12 Proof of coverage **8**

13 What the algorithm does not cover **9**

1 Why an algorithm, and what "every case" means

The first version of the case tree sorted a flip only by what it does to the stored number. That explains the catastrophic faults well, but it says nothing about whether the flip could have happened at all, whether a cheap check would catch it, or whether the hardware would pass it on. The algorithm here adds those stages and keeps the case tree as stage 1.

"Every case is covered" is meant in a strict sense:

1. Every flip has exactly one case. The checks inside each stage run in a fixed order and the first that matches decides, so no flip has two cases and none has zero.
2. This is checked by enumeration, not sampling. For each of the five element formats, every (code, bit) pair is classified: 2,048 pairs for the 8-bit formats, 384 for the 6-bit ones and 64 for `e2m1`. For the scale, all 256 bytes × 8 bits are classified, both for a zero block and for a non-zero one. The test `tests/test_cases.py` asserts that each one maps to a single label through `mxfi.cases.label`.
3. Cases that cannot happen in a clean tensor are still covered. A flip of a value that is already NaN (S7, E7) only happens after an earlier fault, but persistent weight faults do pile up between reloads, so the algorithm has to say what happens.

2 Background: what is stored

An MX tensor is cut into blocks of $K = 32$ values. Each block stores one shared 8-bit scale byte X in E8M0 format and K element codes in a small floating-point format. The value of element i is

$$x_i \;=\; 2^{X-127} \cdot v(c_i),$$

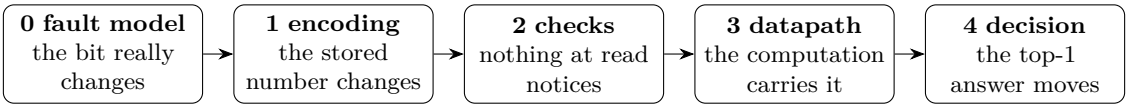
where $v(c)$ is the value of element code c. E8M0 has no sign and no mantissa: bytes 0–254 are the powers 2^{-127} to 2^{127}, and byte 255 is NaN. The OCP rule chooses $X = \lfloor \log_2 \max_i |x_i| \rfloor - e_{\max} + 127$, so the largest element of every non-zero block lands in the format's top binade. Table 1 lists the element formats.

Table 1: Element formats. "Specials" are codes that decode to NaN or ±Inf.

format	bits	bias	$e_{\max}$	largest value	smallest subnormal	special codes
`e4m3` (FP8)	8	7	8	448	2^{-9}	±NaN = S.1111.111
`e5m2` (FP8)	8	15	15	57,344	2^{-16}	±Inf, 6 NaNs (exponent 11111)
`e3m2` (FP6)	6	3	4	28	2^{-4}	none
`e2m3` (FP6)	6	1	2	7.5	2^{-3}	none
`e2m1` (FP4)	4	1	2	6	2^{-1}	none

Every element code belongs to one *value class*: zero, subnormal (exponent field 0, mantissa non-zero), normal, top binade (the largest finite exponent field), max (the largest finite magnitude) or special. The class matters because the effect of a flip depends on where in the range the value sits, not only on which bit flips.

3 The five-stage chain



A flip can stop at any stage. A stuck-at-1 fault on a bit that already holds 1 stops at stage 0; a sign flip of +0 stops at stage 1; a NaN element code that a reader decodes as 0 stops at stage 2; a NaN zeroed by a ReLU stops in stage 3 after erasing one channel; and a small finite error usually stops at stage 4, because the prediction does not change. Stage 5 then adds up the damage over all faults.

4 The algorithm

Algorithm 1 is the whole procedure. Algorithms 2 and 3 are the two case trees of stage 1, written out in their checking order.

5 Stage 0: the fault model

Kinds of fault. The algorithm covers a transient flip $0 \to 1$ (F1) or $1 \to 0$ (F2), a stuck-at-1 (F3) or stuck-at-0 (F4) bit, and several bits of one word at once (F5). A stuck-at fault on a bit that already holds the stuck value changes nothing (F0). So a stuck-at-1 fault behaves exactly like the set of $0 \to 1$ flips, and a stuck-at-0 fault like the $1 \to 0$ flips.

Algorithm 1 ASSESS: what one fault does

Require: fault f (site, bit or bit mask, kind), fault model M, datapath P

Stage 0 — does the bit actually change?

$b \leftarrow$ current value of the bit
if f is stuck-at-v **and** $b = v$ **then return** F0 MASKED
end if
if f hits a padding lane of the last block **then**
  if P drops padding at decode **then return** F0 MASKED
  else continue; the lane meets a 0 input, so only a NaN/Inf code still matters
  end if
end if
$w(f) \leftarrow \text{exposure}(f) \cdot p_M(b \to \neg b)$ ▷ weights always resident; activations only while buffered

Stage 1 — what does the stored number become?

if f is in a scale byte **then**
  case $\leftarrow$ SCALECASE(f) ▷ Alg. 2
else
  case $\leftarrow$ ELEMENTCASE(f) ▷ Alg. 3
end if
record context: b, $|\delta| = |v' - v| \cdot 2^{X-127}$, value class, one flip from NaN?, block maximum?, demotes the only top-binade element?, X' in band?, zero-block scale (127 or 0), partial sum P_B, layer, input activity a

Stage 2 — does anything notice at read time?

if element code is NaN/Inf **then** ▷ D1
  if guard G1 is on **then** decode it as 0 ▷ E1 becomes "one weight zeroed"
  end if
end if
if X' outside the tensor's clean band **then** ▷ D2; D2a for weights: bit 7 set
  if guard G2 is on **and** X' above the band **then** treat the block as 0 ▷ S1/S2/S3 → S6
  end if
end if
if a non-zero block has no top-binade element **then** report; reload weights ▷ D3
end if

Stage 3 — what does the datapath do with it?

if the value is NaN **then**
  zero inputs do not help: $\text{NaN} \cdot 0 = \text{NaN}$
  if a NaN-zeroing ReLU follows **then** only that output channel is erased ▷ P2
  else NaN reaches the output ▷ P1
  end if
else if the value or the block's partial sum is ±Inf **then**
  +Inf passes any ReLU; −Inf is zeroed by any ReLU ▷ P3
  $\text{Inf} \cdot 0 = \text{NaN}$; $(+\text{Inf}) + (-\text{Inf}) = \text{NaN}$
  if the accumulator saturates **then** the error stays finite and silent ▷ P4
  end if
else
  output error $\approx \delta \cdot a$ (element) or $(2^{\Delta} - 1) \cdot P_B$ (scale)
  zero inputs mask it fully (P5); ReLU masks part of a negative error (P6)
end if
if activations are re-quantised **then** spread to block neighbours (P7); a NaN gives a NaN scale and 32 NaNs (P8)
end if

Stage 4 — does the answer change?

outcome $\in$ {O1 no change, O2 silent corruption, O3 non-finite output (caught by D4)}
damage(f) $\leftarrow$ share of images whose top-1 differs from the fault-free run (a non-finite logit row counts as a mismatch)

Stage 5 — aggregate and protect

rate $= \sum_f w(f) \cdot \text{damage}(f)$; silent rate = the part no check D1–D4 catches
protection order = bit positions sorted by damage per stored bit

Algorithm 2 ScaleCase: flip of scale byte X

$X' \leftarrow X \oplus \text{mask}$; $\Delta \leftarrow X' - X$ ▷ block multiplied by 2^{Δ}
if $X' = 255$ **then return** S1 to NaN ▷ whole block NaN, zero blocks too
else if $X = 255$ **then return** S7 from NaN ▷ only after an earlier fault
else if every element is ± 0 **then return** S0 zero block ▷ masked
else if $\max_i |v_i| \cdot 2^{X'-127} > \text{FP32}_{\max}$ **or** $\Delta = +128$ **then return** S2 overflow
else if $\Delta \geq 8$ **then return** S3 grow $\times 256$ or more
else if $\Delta > 0$ **then return** S4 grow $\times 2$, $\times 4$, $\times 16$
else if $\Delta \geq -2$ **then return** S5 shrink $\div 2$, $\div 4$
else return S6 erase ▷ $\div 16$ or more for one bit
end if

Algorithm 3 ElementCase: flip of element code c with value v

$c' \leftarrow c \oplus \text{mask}$; $v' \leftarrow v(c')$
if c is NaN/Inf **then return** E7 from special
else if c' is NaN/Inf **then return** E1 to special ▷ always a $0 \to 1$ flip
else if sign bit **then return** E0 masked if $v = \pm 0$, else E2 negate ▷ $|\delta| = 2|v|$
else if exponent bit $0 \to 1$ **then**
    if $v = 0$ **then return** E3a from zero
    else if v subnormal **then return** E3b subnormal→normal
    else return E3 grow $\times 2^{2^k}$
    end if
else if exponent bit $1 \to 0$ **then**
    if $v' = 0$ **then return** E4a to zero
    else if v' subnormal **then return** E4b normal→subnormal
    else return E4 shrink $\div 2^{2^k}$
    end if
else if mantissa bit $0 \to 1$ **then return** E5a from zero if $v = 0$, else E5 grow
else return E6a to zero if $v' = 0$, else E6 shrink
end if

Why direction matters. Every catastrophic case needs a $0 \to 1$ flip. NaN codes, Inf codes, scale byte 255 and the +128 scale jump are all runs of 1s, so a bit has to be *set* to reach them. In both exhaustive campaigns, all 2,468 (`e3m2`) and 10,321 (`e4m3`) catastrophic faults were $0 \to 1$.

Table 2: Flip direction in the exhaustive ResNet8 weight campaigns (E27).

	`e3m2`	`e4m3`
stored bits that are 0	43%	40%
share of damage from $0 \to 1$ flips (= stuck-at-1)	87.5%	88.7%
per-inference rate, $0 \to 1$ flips only	0.00768	0.02089
per-inference rate, $1 \to 0$ flips only	0.00110	0.00267
ratio	7.0×	7.8×

So the failure rate is not a property of the format alone. A memory whose upsets are mostly $1 \to 0$ would see about 8× fewer failures than the symmetric model assumes. A stuck-at-0 bit is almost harmless; a stuck-at-1 bit is the dangerous kind.

Exposure. Weights stay in memory for the whole deployment, so a weight fault is persistent and hits every inference until the weights are reloaded. An activation is held only while it sits in a buffer, so an activation fault is transient and hits one inference.

Padding lanes. When a tensor's length is not a multiple of 32, the last block is padded. A flip in a padding lane is masked if decode drops the padding, which is what `mxfi` does. Hardware that multiplies the whole block instead pairs the lane with a 0 input. A finite error is still masked, but a NaN or Inf code is not, since $\text{NaN} \cdot 0$ and $\text{Inf} \cdot 0$ are both NaN.

Several bits of one word. With $\Delta = X' - X$, the scale tree holds as written: a mask that gives $\Delta = -3$ is S6 ($\div 8$). For an element, the first two checks (from or to a special code) still hold. When the flipped bits

span fields or flip in both directions, "field" and "direction" no longer mean anything, so the case is described by the value-class change and $|\delta|$ instead. Flips within one word were measured earlier to be sub-additive (E21).

6 Stage 1: the scale case tree

The key fact. Flipping bit j of the scale byte changes X by exactly $+2^j$ if the bit was 0 and -2^j if it was 1. So the block is multiplied by $2^{\pm 2^j}$, and how far it moves is fixed by the bit position alone. The value of X only decides the direction and whether a boundary (255, or the FP32 range) is crossed. For a typical weight scale $X = 117$ (`01110101`):

bit j	0	1	2	3	4	5	6	7
Δ	-1	$+2$	-4	$+8$	-16	-32	-64	$+128$
case	S5	S4	S6	S3	S6	S6	S6	S2

The cases, in checking order.

S1 to NaN ($X' = 255$).
Only bytes with seven 1s can reach 255. The whole block decodes to NaN, zeros included, since $0 \cdot \text{NaN} = \text{NaN}$. That is why this check comes before the zero-block check. In the weight campaigns no clean scale had seven 1s, so S1 never occurred there; it appears in activations, whose scales sit near 127 (0.2–1.0% of activation scale flips in FP6/FP4).

S7 from NaN ($X = 255$).
The block was already NaN after an earlier fault; any flip makes it finite again. This is a repair, not a shrink, so it is its own case.

S0 zero block.
Every element is ± 0, so any finite scale gives 0. Masked.

S2 overflow.
The block leaves the FP32 range, either at decode or (for $\Delta = +128$) in the accumulation that reads it. Whether it really becomes Inf is a stage-3 question; see Section 9.

S3 grow $\times 256$ or more ($\Delta \geq 8$).
Finite but huge: mean damage 0.74–0.76.

S4 grow ($0 < \Delta < 8$).
$\times 2$, $\times 4$ or $\times 16$. Wide spread: the median damage is 0.04–0.06, but the 90th percentile is 0.33–0.43, depending on the block's partial sum P_B.

S5 shrink ($-2 \leq \Delta < 0$).
$\div 2$ or $\div 4$. Mild.

S6 erase ($\Delta \leq -3$).
The block shrinks $16\times$ or more and is effectively deleted.

7 Stage 1: the element case tree

The case of an element flip is the field of the bit, the direction of the flip and the value-class transition it causes. Worked examples in `e4m3` (bias 7):

case	before	bit flipped	after	effect
E3 grow	$1.0 = 0.0111.000$	exponent bit 3, $0 \to 1$	$0.1111.000 = 256$	$\times 2^8$
E4 shrink	$1.0 = 0.0111.000$	exponent bit 2, $1 \to 0$	$0.0011.000 = 1/16$	$\div 2^4$
E4b normal→sub	$1.5 \cdot 2^{-6} = 0.0001.100$	exponent bit 0	$0.0000.100 = 2^{-7}$	$\div 3$
E1 to NaN	$448 = 0.1111.110$	mantissa bit 0	$0.1111.111 = \text{NaN}$	non-finite
E2 negate	v	sign	$-v$	$\lvert\delta\rvert = 2\lvert v\rvert$
E0 masked	$+0$	sign	-0	none

E7 from special.
The code was already NaN/Inf. In a clean tensor this never occurs; in `e5m2` a mantissa flip of an Inf code gives NaN.

E1 to special.
The new code is NaN or Inf. Every such flip is $0 \to 1$. In `e4m3` only the seven magnitudes 1.875, 30, 120, 240, 352, 416 and 448 are one flip from NaN, and both exponent and mantissa bits can do it. In `e5m2` only exponent bits can, and they can reach ±Inf (E1i) as well as NaN (E1n). `e3m2`, `e2m3` and `e2m1` have no special codes, so E1 and E7 never occur in them. Mean damage of E1 in the weight campaign: 1.0 for every single fault.

E0 / E2 sign.
A sign flip of zero is masked; of anything else it negates the value. Where a format puts its special codes is a parameter: in an FNUZ format the -0 pattern 1.0000.000 is NaN, so the sign flip of $+0$ becomes E1.

E3 / E4 exponent.
Flipping exponent bit k between two normal values multiplies or divides by exactly 2^{2^k}. The sub-cases are the boundary crossings: from zero (E3a), subnormal to normal (E3b), to zero (E4a), normal to subnormal (E4b).

E5 / E6 mantissa.
Smaller changes inside one binade, with the zero crossings E5a and E6a.

The case is not enough for finite flips. Table 3 shows how much of the measured damage each description explains (held-out R^2, two-fold). The case decides the catastrophic part almost completely, but within ordinary finite element flips it explains only 2–5%. What matters there is the size of the error reaching the next layer: $|v' - v| \cdot 2^{X-127}$ times the activity of the input the weight multiplies. The block's scale X is a factor the first version ignored.

Table 3: Variance of damage explained (E27).

model	all faults (`e3m2` / `e4m3`)	finite element flips (`e3m2` / `e4m3`)
site only	0.22 / 0.09	0 / –
+ case	0.91 / 0.97	0.02 / 0.05
+ bit position	0.92 / 0.98	0.02 / 0.06
+ layer	0.94 / 0.98	0.19 / 0.21
+ size of the weight error	0.94 / 0.98	0.54 / 0.54
size × input activity instead	0.94 / 0.98	0.67 / 0.64
+ sign of the error	0.94 / 0.98	0.70 / 0.66

Context recorded with each case. Flip direction; value class; one flip from NaN; whether the element is its block's maximum (5% of `e3m2` element faults, 21% of their damage); whether it demotes the block's only top-binade element; whether X' is in the clean band; the zero-block scale encoding (a from-zero flip in a block with $X = 127$ gives a value up to ±2, but with $X = 0$ it gives about 2^{-126}); the special-code placement; layer; error size; input activity.

Measured damage by case. Table 4 gives the exhaustive weight results.

Activations behave differently. Counting over real inputs of ResNet8 and RepVGG-A0: 40–55% of activation values are exactly 0, so 5–15% of element flips are masked sign flips and 33–48% are flips from zero to a tiny value. In OCP `e4m3` 0.7–1.2% of activation element flips reach NaN (3.5–5.2% in `e5m2`). With FNUZ-style placement it would be 5.0–6.8%, because zero is the most common value and its sign flip would be NaN.

8 Stage 2: free checks and guards

A clean OCP MX tensor obeys three invariants, and checking them needs no extra memory:

D1 No element code is NaN/Inf. Guard **G1**: decode such a code as 0.

Table 4: Mean damage per case, exhaustive ResNet8 weight campaigns (E25). Damage share is the part of the total per-inference rate the case causes.

		e3m2		e4m3	
label	case	mean damage	damage share	mean damage	damage share
S2	scale overflow	0.974	56.5%	0.978	16.0%
S3	scale grow large	0.742	10.4%	0.756	12.4%
S4	scale grow small	0.146	14.6%	0.111	2.2%
S5	scale shrink	0.011	0.7%	0.017	0.3%
S6	scale erase	0.024	5.5%	0.029	1.8%
E1	element to NaN	–	–	1.000	52.2%
E2	negate	0.0030	5.5%	0.0073	3.8%
E3	exponent grow	0.0017	3.1%	0.0055	2.9%
E4	exponent shrink	0.0010	3.2%	0.0043	6.3%
E5	mantissa grow	0.0002	0.3%	0.0015	1.2%
E6	mantissa shrink	0.0001	0.2%	0.0012	0.9%
E0, E3–6a/b	boundary cases	≤0.00013	<0.1%	≤0.00013	<0.1%

D2 Every scale lies in the tensor's clean band (known offline for weights, calibrated on a few inputs for activations). For weights, D2a is simpler: no clean weight scale has bit 7 set. Guard **G2**: a block whose scale is above the band is treated as 0.

D3 Every non-zero block has an element in the top binade.

D4, a check that the logits are finite, sits at the output and catches what reaches it as NaN/Inf.

Table 5: Share of damage each check flags, and the effect of the guards (E27).

	e3m2	e4m3
D1 element code is NaN/Inf	–	52.2%
D2a scale bit 7 set	56.5%	16.0%
D2b scale outside the clean band	85.7%	32.1%
D3 lost top-binade element	0.1%	0.2%
D1 + D2b + D3 at read (3.5% / 4.1% of faults)	85.8%	84.5%
D4 non-finite output	56.2%	68.1%
rate, no guard	0.00878	0.0236
rate, G2 only	0.00206 (4.3×)	0.0168 (1.4×)
rate, G1 only	–	0.0113 (2.1×)
rate, G1 + G2	0.00206 (4.3×)	0.00454 (5.2×)
silent rate left after all checks	0.00124	0.00364

G2's effect is exact: a block treated as 0 has the measured damage of the same block's $\div 2^{64}$ flip. G1 was measured by injecting 2,000 E1 faults with the code decoded as 0; mean damage fell from 1.0 to 0.0064. For activations, a band calibrated on 10 images had 0.01% false alarms on 10 others and caught every flip of scale bits 3–7.

9 Stage 3: the datapath

The same flip can be catastrophic or mild depending on the hardware. The rules:

P1 / P2 NaN.

$\text{NaN} \cdot 0 = \text{NaN}$, so zero inputs do not mask a NaN weight. The PyTorch ReLU passes NaN on (P1). A NaN-zeroing ReLU ($x > 0 \ ? \ x : 0$, IEEE maxNum) turns it into "this output channel is erased" (P2), except in the last layer, where no ReLU follows. Re-injected: E1 mean damage 1.00 with a NaN-passing ReLU, 0.085 with a NaN-zeroing one, 0.0064 with the read guard G1.

P3 overflow.
+Inf passes any ReLU; −Inf is zeroed by any ReLU (a batch norm with negative γ can flip the sign first). $\text{Inf} \cdot 0 = \text{NaN}$, and +Inf and −Inf meeting in a later sum give NaN.

S2 in detail.
$\Delta = +128$ multiplies the block's partial sum by 2^{128}, which overflows FP32 only when $|P_B| > 1$. In both campaigns 98.7% of these flips gave non-finite outputs, not 100%. A negative partial sum (−Inf, zeroed by ReLU) and a partial sum below 1 can both leave the output finite; the two were not separated. Under a NaN-zeroing ReLU S2's damage stays at 0.97, because +Inf is not NaN, so only a scale check (D2/G2) helps.

P4 saturation.
A saturating accumulator turns overflow into a silent finite error.

P5 zero inputs.
A finite weight error that only multiplies zeros changes nothing. (ResNet8 had no input channel that was zero on all 200 images, so this was not observed.)

P6 ReLU asymmetry.
In hidden layers, an error that raises the pre-activation is 1.07–1.12× worse than one that lowers it, up to 1.2–1.4× at the largest errors, because a positive error can switch a unit on.

P7 re-quantisation.
In an all-MX pipeline every layer's input is re-encoded. An error that makes a value $r\times$ its block maximum raises the block's scale, and neighbours far below the new maximum are flushed to 0. Share of non-zero neighbours flushed:

format	$r = 2$	$r = 4$	$r = 16$	$r = 256$	$r = 65536$
`e5m2`	0	0	0	0	0
`e4m3`	0	0	0.01%	0.1%	29%
`e3m2`	0.5%	0.9%	3.9%	55%	100%
`e2m3`	3.7%	7.4%	28%	100%	100%
`e2m1`	15%	29%	91%	100%	100%

P8 NaN at re-quantisation.
A NaN reaching re-quantisation becomes a NaN scale and so 32 NaNs. What an Inf does there depends on the implementation (a NaN scale, or saturation to the largest code).

10 Stage 4: the decision

Damage is the share of test images whose top-1 prediction differs from the fault-free run. An image whose logit row is not finite is counted as class −1, a mismatch, so a NaN that reaches every image gives damage exactly 1. Each fault ends in one of three outcomes:

outcome	`e3m2` rate (share)	`e4m3` rate (share)
O1 no change	0	0
O2 silent corruption	0.00384 (44%)	0.00750 (32%)
O3 non-finite output, caught by D4	0.00494 (56%)	0.01606 (68%)

For a weight fault one caught NaN output is enough: the fault is persistent, so the system can reload the weights, and the fault becomes a detected error instead of a silent one. Most of `e4m3`'s extra damage over `e3m2` is this detectable kind.

11 Stage 5: aggregate and protect

The per-inference rate is $\sum_f w(f) \cdot \text{damage}(f)$, and its silent part is what no check D1–D4 catches. Sorting bit positions by damage per stored bit gives the protection order:

- `e3m2`: scale bit 7 is 0.5% of storage and 57% of damage. Scale bits 7, 3, 2 and 1 are 2% of storage and 83% of damage, so protecting by bit position works well.

- `e4m3`: 27% of storage is needed for half the damage, because NaN damage is spread over every element bit. Protecting by bit position works badly; G1 plus G2 do far better at no storage cost.

12 Proof of coverage

Table 6 counts how many (code, bit) pairs of each format fall in each element case. Every row sums to the full code space ($2^{\text{width}} \times \text{width}$). For the scale, the 4,096 (byte, bit, zero block or not) combinations split into S0 2,032, S1 16, S2 127, S3 508, S4 381, S5 254, S6 762 and S7 16. All eight scale cases and all fourteen element labels occur, and none is left over.

Table 6: Exhaustive enumeration of element flips by case.

format	E0	E1	E2	E3	E3a	E3b	E4	E4a	E4b	E5	E5a	E6	E6a	E7	total
`e4m3`	2	14	252	440	8	56	440	8	56	372	6	372	6	16	2048
`e5m2`	2	40	246	560	10	30	560	10	30	244	4	244	4	64	2048
`e3m2`	2	0	62	72	6	18	72	6	18	60	4	60	4	0	384
`e2m3`	2	0	62	32	4	28	32	4	28	90	6	90	6	0	384
`e2m1`	2	0	14	8	4	4	8	4	4	6	2	6	2	0	64

Table 7: All cases, by stage.

stage	cases
0 fault model	F0 masked (stuck value already stored, or padding dropped) · F1 transient $0 \to 1$ · F2 transient $1 \to 0$ · F3 stuck-at-1 · F4 stuck-at-0 · F5 several bits of one word
1 scale	S0 zero block · S1 to NaN · S2 overflow · S3 grow $\geq \times 256$ · S4 grow $\times 2/ \times 4/ \times 16$ · S5 shrink $\div 2/ \div 4$ · S6 erase · S7 from NaN
1 element	E0 masked · E1 to NaN/Inf · E2 negate · E3 exponent grow (E3a, E3b) · E4 exponent shrink (E4a, E4b) · E5 mantissa grow (E5a) · E6 mantissa shrink (E6a) · E7 from NaN/Inf
2 checks	D1 special code $\to$ G1 · D2 scale out of band (D2a bit 7) $\to$ G2 · D3 lost block maximum · D4 output check
3 datapath	P1 NaN propagates · P2 NaN zeroed by ReLU · P3 overflow to $\pm$Inf · P4 saturation · P5 zero inputs mask finite errors · P6 ReLU asymmetry · P7 re-quantisation spreads errors · P8 NaN block $\to$ 32 NaNs
4 decision	O1 no change · O2 silent corruption · O3 non-finite output

13 What the algorithm does not cover

- Faults outside stored MX values: accumulators, partial sums, biases, normalisation parameters, control logic and addressing (a block paired with the wrong scale). The element tree is format-generic, so FP32/BF16 values could use it with their own special codes, but this was not measured.
- Faults piling up across words in weight memory between reloads (S7 and E7 say what one more flip does, but the build-up itself was not measured).
- Damage under a saturating accumulator, an FP16 accumulator, or a full MX activation pipeline end to end. P3, P4, P7 and P8 are rules from the arithmetic here, not injected measurements, and the same holds for the $\pm$Inf and padding-lane rules.
- One network measured exhaustively (ResNet8, width 16). Activation results cover ResNet8 and RepVGG-A0.

Reproducing. Code is in `mxfi/cases.py`; experiments in `experiments/e25`–`e28`; raw output in `results/`.

```
python -m pytest tests/test_cases.py -q     # the coverage test
python -m experiments.e25_flip_cases
python -m experiments.e26_datapath          # about 20 min on CPU
python -m experiments.e27_fault_model       # after e25 and e26
python -m experiments.e28_activation_cases
```